

Wireshark Packet Analysis Lab

Network Traffic Capture & Port Scan Detection — Personal Security Lab

1. Objective

This lab documents the use of Wireshark to capture and analyze live network traffic on a personal Windows machine, with the goal of understanding packet structure, protocol behavior, and the network signature of a TCP port scan. A custom-built Python port scanner (developed separately) was used to generate real scan traffic for analysis, simulating reconnaissance behavior used in penetration testing.

2. Environment & Tools

Tool	Wireshark v4.6.6
OS	Windows
Capture Interface	Wi-Fi adapter
Scanner used	Custom Python TCP port scanner (multithreaded)
Targets	Local machine (172.16.3.181), Default gateway / router (172.16.1.1)

3. Initial Capture & Interface Orientation

A live capture was started on the Wi-Fi interface to observe baseline traffic. Wireshark's three-panel layout was used throughout this lab:

- Packet list (top) — one row per captured packet: number, timestamp, source, destination, protocol, length, and info.
- Packet details (middle) — protocol stack breakdown of the selected packet (Frame -> Ethernet -> IP -> Transport -> Application layer).
- Hex dump (bottom) — raw bytes of the packet in hexadecimal and ASCII.

Baseline traffic consisted mostly of MDNS (Multicast DNS) broadcasts from local devices advertising services such as KDE Connect, Google Cast, and AirPlay — normal background activity on a home network.

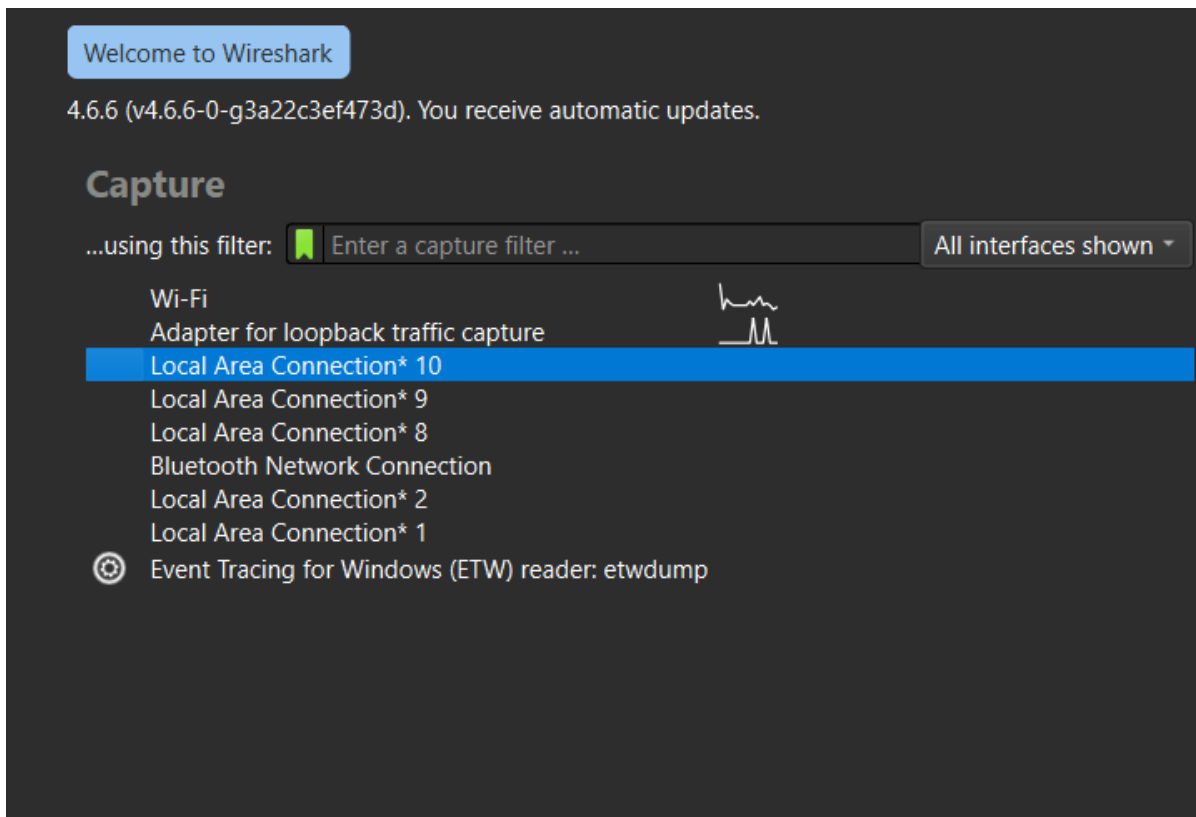


Figure 1 — Wireshark interface selection screen showing available capture interfaces, with Wi-Fi showing active traffic.

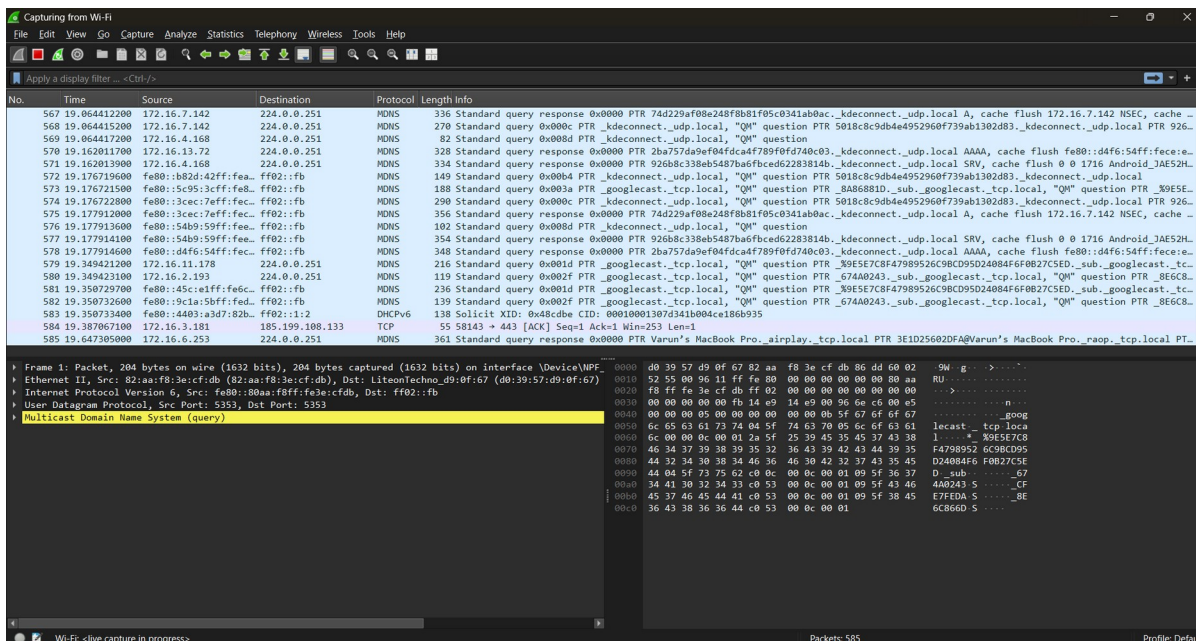


Figure 2 — Live capture on Wi-Fi interface showing the three-panel layout: packet list, packet details, and hex dump.

4. Display Filters

Display filters were used to isolate specific protocols from the full capture:

dns

Showed DNS query/response pairs — e.g. a query for `a-api.anthropic.com` resolved to `160.79.104.10`. This demonstrates how domain names are translated to IP addresses before a connection is made.

tcp

Showed TCP-layer packets including handshakes (SYN, SYN-ACK, ACK), data transfer, and connection teardown (FIN, ACK and RST, ACK).

https

Captured an HTTP GET request to a captive-portal check endpoint (/204) returning a 307 Temporary Redirect, alongside encrypted TLSv1.2 Application Data to 8.8.8.8 — illustrating the difference between cleartext HTTP and encrypted HTTPS traffic at the packet level.

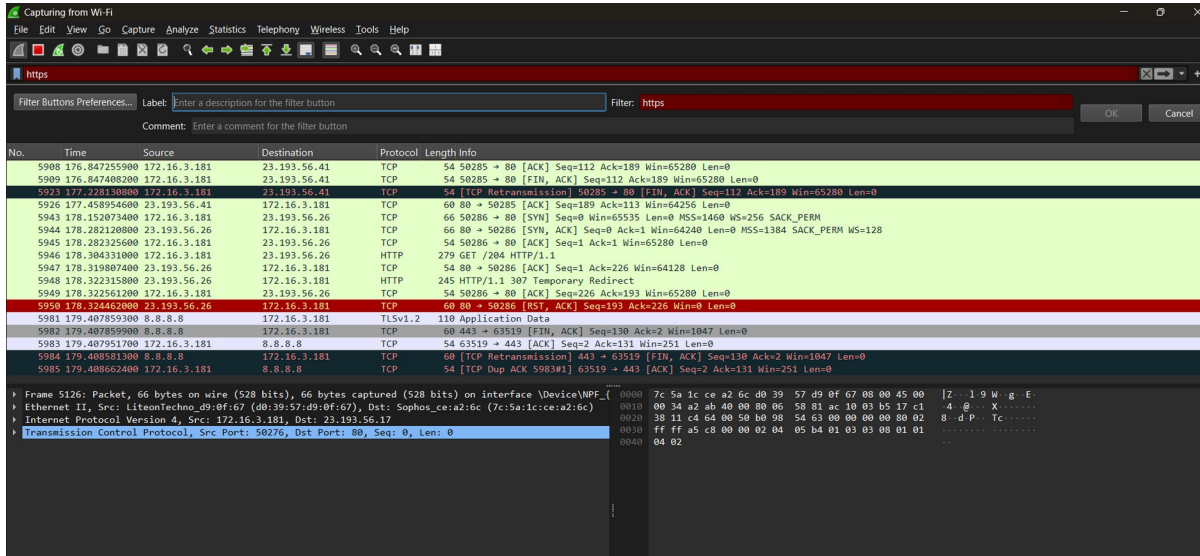


Figure 3 — TCP/HTTPS filtered traffic showing an HTTP 307 redirect and encrypted TLSv1.2 application data.

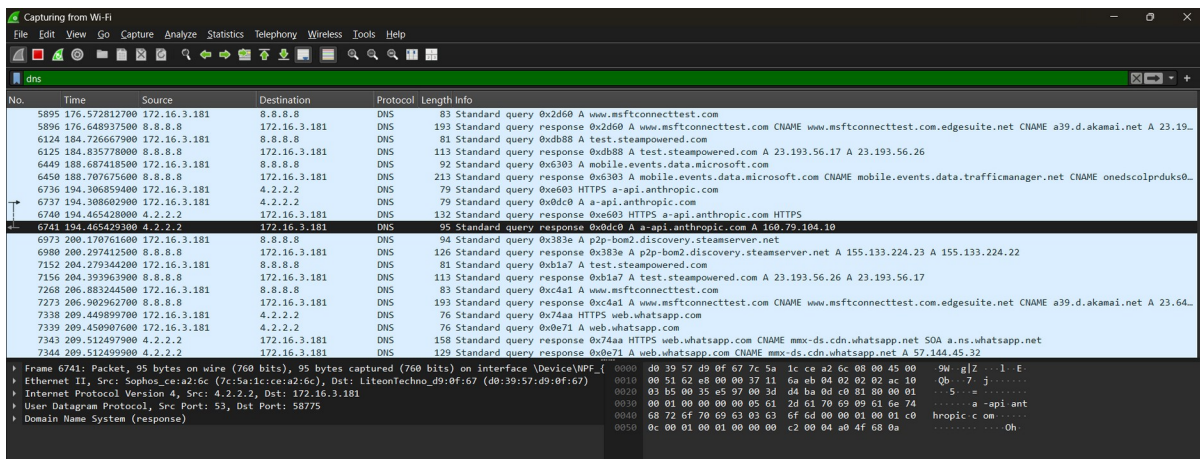


Figure 4 — DNS filter showing query/response pairs, including resolution of a-api.anthropic.com to its IP address.

5. TCP Stream & Protocol Hierarchy

TCP conversations were inspected to understand connection state changes:

- [SYN] — client requests a new connection.
- [SYN, ACK] — server acknowledges and accepts.
- [ACK] — client confirms; connection established.
- [FIN, ACK] / [RST, ACK] — connection closed gracefully or forcefully reset.

An out-of-order SYN-ACK packet was also observed, which Wireshark flagged automatically. This was identified as normal network jitter rather than a security concern, since sequence numbers and acknowledgments remained consistent.

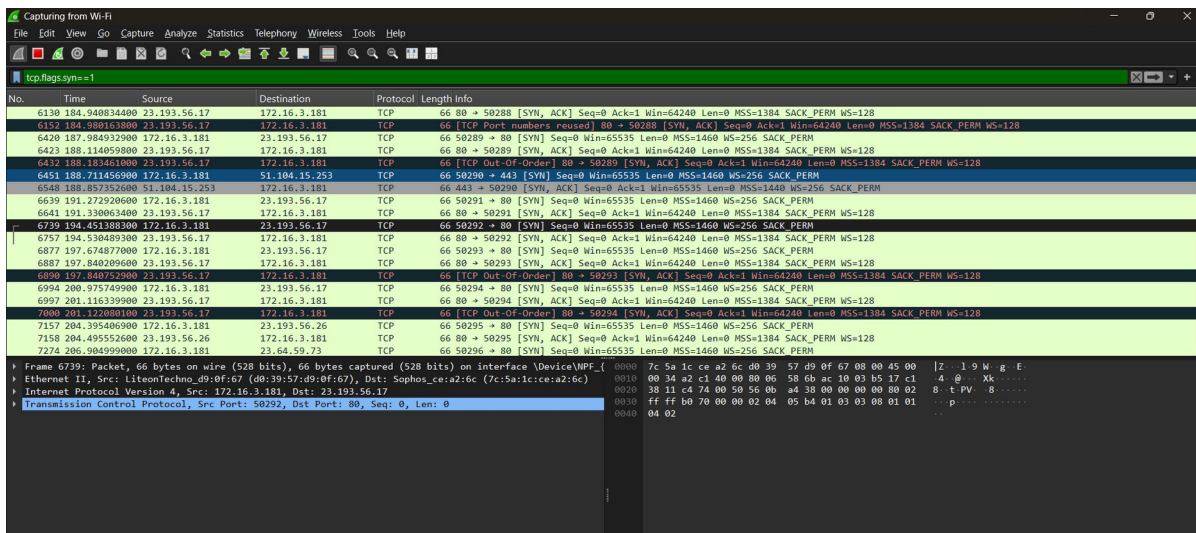


Figure 5 — Baseline SYN traffic filter (`tcp.flags.syn==1`) showing normal repeated connections to a single web server on ports 80/443, used as a comparison point against the port scan evidence below.

6. Port Scan Detection (Core Exercise)

6.1 Methodology

To generate real port scan traffic for analysis, a custom multithreaded Python port scanner was run against the network's default gateway (172.16.1.1) while Wireshark actively captured traffic on the Wi-Fi interface. The scanner attempted TCP connections to all 1024 ports (1-1024) nearly simultaneously using Python's threading module.

Note: an initial attempt to scan localhost (127.0.0.1) produced no visible Wireshark traffic. This is expected — loopback connections to a machine's own IP are handled internally by the OS network stack and do not traverse the physical network adapter, so they are invisible to an interface-level capture. Scanning a real network host (the router) resolved this.

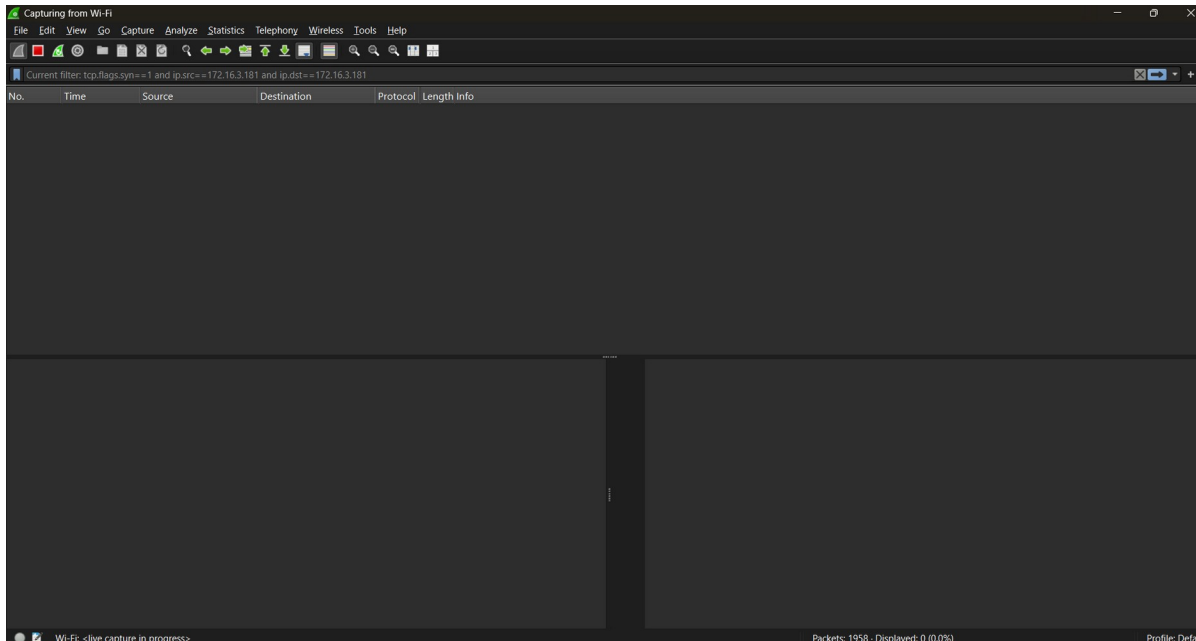


Figure 6 — Confirmed live capture on Wi-Fi interface (1958 packets captured) immediately before running the port scanner against the router.

6.2 Filter Applied

`ip.dst==172.16.1.1 and tcp.flags.syn==1 and tcp.flags.ack==0`

This filter isolates only outbound SYN packets (connection attempts) directed at the target, excluding SYN-ACK replies — showing exactly what the scanning machine transmitted.

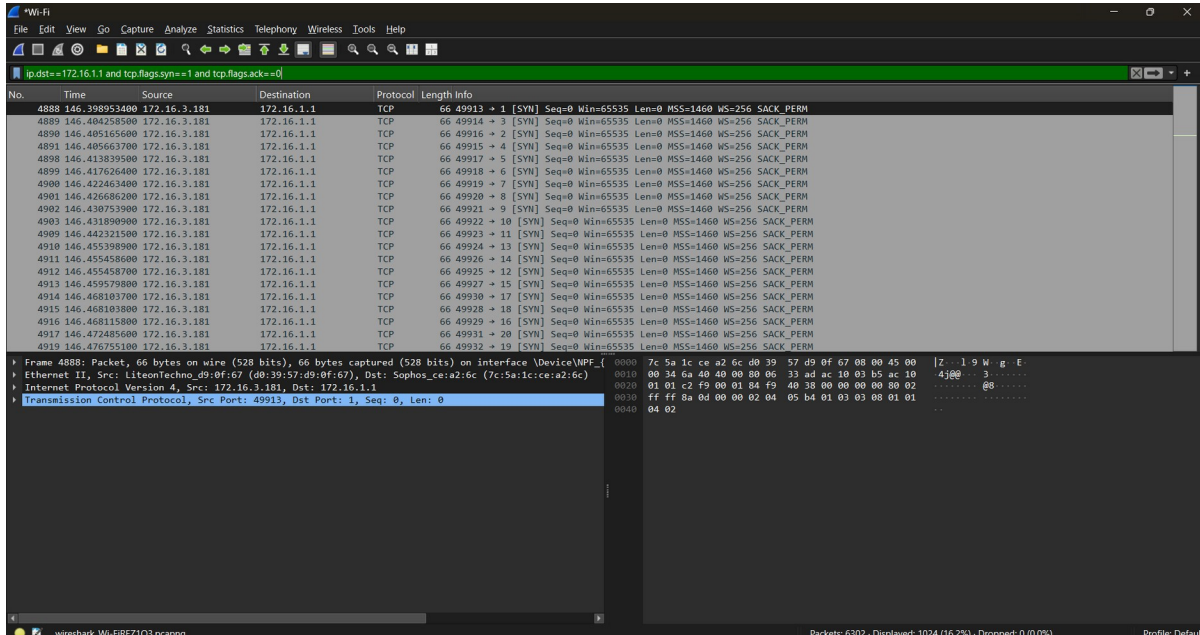


Figure 7 — Port scan evidence: 1024 SYN packets from 172.16.3.181 to 172.16.1.1 across nearly every port (1-1024), captured in under 100 milliseconds. This is the core evidence of the lab.

6.3 Observations

Packets captured	1024 (one SYN per port)
Source IP	172.16.3.181 (scanning machine)
Destination IP	172.16.1.1 (router, single fixed target)
Destination ports	Sequential range 1-1024, arriving in near-random order
Time span	~146.39s to ~146.48s (under 100 milliseconds total)
Source ports	Randomized ephemeral ports (e.g. 49913, 49914, 49916...)

Sample packets from the capture:

```
49913 -> 1 [SYN] Seq=0 Win=65535 MSS=1460 49914 -> 3 [SYN] Seq=0 Win=65535
MSS=1460 49916 -> 2 [SYN] Seq=0 Win=65535 MSS=1460 49915 -> 4 [SYN] Seq=0
Win=65535 MSS=1460
```

6.4 Why This Is Identifiable as a Port Scan

- Single source IP contacting a single destination IP across nearly every port number in sequence — normal traffic targets one or two known ports (e.g. 80, 443), not a full range.
- Extremely high packet rate — 1024 connection attempts completed in under 100ms, far faster than any human-driven or normal application traffic.
- All packets are SYN-only with no corresponding application data — consistent with reconnaissance/enumeration rather than genuine service use.
- Slightly out-of-order port sequence (1, 3, 2, 4...) reflects concurrent multithreaded connection attempts rather than a sequential, single-threaded scan.

This pattern matches the signature that intrusion detection systems (IDS) and SIEM tools use to flag port scanning activity in real network environments — a single host generating a burst of SYN packets across many ports on one target in a very short time window.

7. Conclusion

This lab provided hands-on experience with packet-level network analysis using Wireshark, including protocol filtering, TCP handshake interpretation, and DNS resolution tracing. Most significantly, it demonstrated the complete reconnaissance lifecycle from an offensive security perspective: building a custom port scanning tool in Python, executing it against a live target, and analyzing the resulting network traffic to identify the precise packet-level signature of a port scan. This exercise reinforced the relationship between offensive tooling (the scanner) and defensive detection (traffic analysis) — two complementary skill sets in cybersecurity.

8. Files & Evidence

- `port_scan_evidence.pcapng` — saved Wireshark capture containing the full scan traffic
- `port_scanner_fast.py` — multithreaded Python port scanner used to generate the scan
- Screenshots — packet list views for DNS, TCP, HTTPS filters and the isolated port scan capture